

Appunti — Modulo 3B: Gestione dei Pacchetti Software

Corso: Lab Amministrazione di Sistemi T **Data pratica:** 28/04/2026 **Stato:** ☐
Completato — tutti gli esercizi eseguiti sulla VM

1. Il Problema della Gestione del Software

Ogni software ha un **ciclo di vita**: installazione → aggiornamento → disinstallazione.

Le problematiche che rendono non banale questo ciclo: - **Prerequisiti hardware/SO**: il software deve girare sull'architettura giusta - **Dipendenze da altri componenti software**: librerie, tool di sistema - **Configurazione**: ogni software ha parametri specifici

Installazione manuale

Da binari: copia manuale nei percorsi corretti, verifica manuale compatibilità architettura e dipendenze.

Da sorgente (caso tipico: archivio .tar.gz, codice C, build via autoconf):

```
cd /usr/local/src
tar tvzf net-snmp-5.4.tar.gz      # lista contenuti prima di estrarre
tar xvzf net-snmp-5.4.tar.gz     # estrae
cd net-snmp-5.4/
./configure --help               # esplora opzioni disponibili
./configure [opzioni scelte]     # configura (genera Makefile)
make                             # compila — NON come root
sudo make install                # installa — qui serve root
```

Regola di sicurezza: solo `sudo make install` richiede root. Compilare come root è cattiva prassi: esistono casi di malware che sfruttano sorgenti malevoli scaricati ed eseguiti come root.

Autenticità del software scaricato

Prima di estrarre un archivio, verificarne l'autenticità:

```
gpg --verify FILE.asc FILE.tar.gz # mostra il key id della firma
gpg --keyserver pgpkeys.mit.edu --recv-key <KEY_ID>
gpg --verify FILE.asc FILE.tar.gz # ripetere con la chiave recuperata
```

```
# Alternativa con fingerprint:  
sha256 -c FILE.sha256
```

Riflessioni sull'installazione da sorgente

L'installazione da sorgente è utile in teoria (puoi verificare il codice), ma nella pratica: - Più difficile da mantenere - Richiede molti componenti ausiliari (header, librerie, compilatori) che **possono essere vettori di attacco**

Questo è il motivo per cui esistono distribuzioni e pacchetti: le chiavi di verifica si installano una volta sola, le dipendenze si risolvono automaticamente, la compatibilità binaria tra tutti gli elementi del set è garantita.

2. Distribuzioni e Pacchetti

Domanda: cosa si intende per “distribuzione e pacchetti”?
Sono gli stessi archivi compressi di cui si parlava prima come sorgenti?

No — sono cose diverse. Un **pacchetto** (.deb su Debian, .rpm su Red Hat) contiene il software già **precompilato**: non devi compilare nulla. È un file singolo che include: - i binari pronti all'uso - metadati: dipendenze, architettura supportata, versione del pacchetto - script di pre e post-installazione (es. creare un utente di sistema, generare una chiave)

Una **distribuzione** (Debian, Ubuntu, Fedora...) è una raccolta curata di migliaia di questi pacchetti, firmati dai maintainer, compatibili tra loro per una data versione. Scegliere Debian significa affidarsi a questa curatela — non compilare nulla da sorgente.

Cos'è un pacchetto

Un file singolo che contiene in forma compatta: - Software precompilato - Criteri per la verifica della compatibilità e dei prerequisiti - Procedure di pre/post-installazione

La garanzia di compatibilità richiede di vincolare tre parametri: - **Architettura** (amd64, i386, arm64, ...) - **Versione della distribuzione** (es. bookworm, jammy, ...) - **Versione del software** contenuto

Formato dei nomi

Debian/Ubuntu → formato .deb:

aptitude-0.2.15.9-2_i386.deb
↑nome ↑ver.sw ↑ver.pkg ↑arch

RedHat/CentOS/Fedora → formato .rpm:

httpd-2.4.6-45.el7.centos.x86_64.rpm

Criteri per la scelta di una distribuzione

Criterio	Descrizione
Architetture supportate	Quasi tutte supportano amd64; per architetture esotiche verificare
Stabilità vs aggiornamento	Distro “stabili” includono solo sw collaudato; altre preferiscono novità anche a costo di minor robustezza
Versioned vs rolling	Versioned: aggiornamenti correttivi solo durante il ciclo; rolling: sempre all’ultima versione
LTS	Per server: supporto 5/7 anni con solo security backport
Ampiezza pacchetti	Da 1500 (minimali) a 26000+ (Debian)

Debian (la nostra VM): versioned, orientata alla stabilità, ~26000 pacchetti.

3. Due Famiglie: Debian e Red Hat

Quasi tutte le distro Linux derivano da una di queste due.

Entrambe usano una struttura a tre livelli: 1. **Tool di basso livello** per gestire singoli pacchetti (dpkg / rpm) 2. **Tool intermedi** per gestione coordinata di pacchetti e dipendenze (apt / yum/dnf) 3. **Tool per il reperimento automatico** da repository

4. Dipendenze: il Grafo

Domanda: “DIPENDENZE” — si intendono dipendenze tra software che ne richiedono un altro preinstallato?

Esatto. Le dipendenze possono essere di due tipi: - **Logiche**: un software non ha senso senza un altro (es. un’estensione PHP non serve senza un web server) - **Fisiche**: un binario è compilato usando una libreria e non può girare senza di essa (linking dinamico)

Il package manager conosce l'**intero grafo** di queste relazioni e lo risolve automaticamente: quando installi A, installa anche B e C se necessari; quando rimuovi A, segnala o rimuove B e C se sono diventati orfani.

Pacchetti base e pacchetti -dev

Per ogni libreria esistono due pacchetti distinti:

Pacchetto	Contiene	Usato per
zlib1g	.so (linking dinamico)	Eseguire programmi che usano zlib
zlib1g-dev	.a + header .h (linking statico)	Compilare programmi che usano zlib

Su sistemi deb: suffisso -dev. Su sistemi rpm: suffisso -devel. I sistemi non usati per sviluppo non installano i -dev.

Verificare le dipendenze dinamiche di un binario

```
ldd /usr/sbin/sshd
# output: lista di librerie .so con path e indirizzo in memoria
# es: libz.so.1 => /usr/lib/libz.so.1 (0xb7d63000)
```

Domanda: cosa cambia tra libreria dinamica e statica?

- **Statica** (.a): il codice della libreria viene copiato dentro il binario al momento della compilazione. Il binario è autosufficiente ma più grande.
- **Dinamica** (.so): il codice della libreria rimane separato. Il binario, quando viene eseguito, la carica in memoria dal path indicato da ldd. Più leggero, ma richiede che la libreria sia presente sul sistema.

Su Linux quasi tutti i programmi usano linking dinamico — per questo `ldd /usr/sbin/sshd` mostra un elenco di .so. Se una di quelle librerie viene rimossa o aggiornata a versione incompatibile, `sshd` smette di funzionare.

5. Repository

I pacchetti vengono distribuiti tramite **repository**: raccolte indicizzate di pacchetti, online o su filesystem locali.

Il package manager legge per ogni repo l'indice e i metadati: - conosce quali versioni sono disponibili - conosce le dipendenze tra pacchetti

Configurazione repo in Debian/apt

Domanda: “non ho capito la configurazione repo in Debian/apt, ho installato pacchetti su Debian ma non mi sembra di aver fatto così”

La configurazione è già presente nella VM e non l'hai dovuta toccare — per questo non ti sembra di averla fatta. I file sono già lì e apt li legge silenziosamente ogni volta che usi `apt update`. La riga `deb http://... bookworm main` dice ad apt: *vai a questo URL, cerca i pacchetti per la versione bookworm, nel componente “main”*. In pratica la tocchi solo quando aggiungi un repository esterno (es. Docker, VirtualBox) o quando configuri un server che deve usare un mirror locale.

File principale: `/etc/apt/sources.list` Frammenti aggiuntivi: `/etc/apt/sources.list.d/`

Formato di una riga:

```
deb http://archive.ubuntu.com/ubuntu bionic-updates universe
```

Per aggiungere un repository di terze parti:

Creare il file:

/etc/apt/sources.list.d/virtualbox.list

```
deb http://download.virtualbox.org/virtualbox/debian xenial contrib
```

Il database locale di apt/dpkg è in `/var/lib/dpkg/` e `/var/lib/apt/`.

6. Comandi dpkg e apt su Debian

Tabella comandi principali

Operazione	Comando dpkg	Comando apt
Update indice repo	—	<code>apt update</code>
Cercare pacchetto	—	<code>apt search <keyword></code>
Installare da repo	—	<code>apt install <nome></code>
Installare da file .deb	<code>dpkg -i file.deb</code>	—
Aggiornare tutto	—	<code>apt upgrade</code>
Aggiornare un pacchetto	—	<code>apt upgrade <nome></code>
Rimuovere (mantieni config)	<code>dpkg -r <nome></code>	<code>apt remove <nome></code>
Rimuovere (+ config)	<code>dpkg -P <nome></code>	<code>apt purge <nome></code>
Rimuovere dipendenze orfane	—	<code>apt autoremove</code>

Operazione	Comando dpkg	Comando apt
Info pacchetto	<code>dpkg -s <nome></code>	<code>apt show <nome></code>
Lista file installati	<code>dpkg -L <nome></code>	—
A quale pkg appartiene un file	<code>dpkg -S /path/file</code>	—
Lista pacchetti installati	<code>dpkg -l</code>	<code>apt list --installed</code>
Lista aggiornabili	—	<code>apt list --upgradable</code>
Provenienza pacchetto	—	<code>apt-cache showpkg <nome></code>

apt remove vs apt purge

- `apt remove`: rimuove i binari, **mantiene** i file di configurazione in `/etc/`
- `apt purge`: rimuove tutto inclusi i file di configurazione
- `apt autoremove`: rimuove i pacchetti installati come dipendenze che non servono più

Nota pratica: se vuoi reinstallare un pacchetto da zero (es. server con configurazione corrotta), usa `purge`, non `remove`. Con `remove` la vecchia configurazione rimane e influenzerà la nuova installazione.

7. Autenticità dei Pacchetti (sistema distribuzioni)

Domanda: “come funziona questa autenticazione dei pacchetti? come si garantisce che la chiave non sia recapitata da malintenzionati?”

Nelle distribuzioni il problema della distribuzione sicura delle chiavi è risolto **una volta sola al momento dell’installazione del sistema operativo**: il media di installazione (ISO, USB) contiene già le chiavi pubbliche dei maintainer, verificate fuori banda (es. scaricando la ISO da un mirror ufficiale e verificandone il checksum sha256). Da quel momento: 1. Il repo distribuisce i pacchetti firmati con la propria chiave privata 2. `apt` verifica ogni pacchetto scaricato con la chiave pubblica già presente in `/etc/apt/trusted.gpg.d/` 3. Se la firma non quadra, `apt` rifiuta il pacchetto

Il rischio residuo è quello che descrivi: un attaccante che compromette il repo e sostituisce la chiave pubblica sul tuo sistema. Per questo `/etc/apt/trusted.gpg.d/` è accessibile solo da root. La cosa più importante è non aggiungere repo di terze parti senza verificare la provenienza della chiave.

I keyring `.deb` vanno in `/etc/apt/trusted.gpg.d/`:

```
# Aggiungere la chiave pubblica di un repo (metodo moderno):  
gpg -o /etc/apt/trusted.gpg.d/mio-repo.gpg --dearmor pubkey.txt
```

Nota: apt-key (il vecchio metodo) è **deprecato** — non usarlo.

8. Gestire la Provenienza dei Pacchetti (Software Injection)

□ Questa sezione non era presente negli appunti grezzi.

Rischio: se un repo di terze parti contiene un pacchetto con lo stesso nome di un pacchetto di sistema e lo dichiara a versione più recente, apt lo installerà al posto di quello ufficiale → **software injection** (rilevante per Security, Supply Chain Attacks).

```
# Verificare da quale repo proviene un pacchetto installato:  
apt-cache showpkg <nome>
```

Version pinning (bloccare un pacchetto a una versione specifica): - Editare /etc/apt/preferences.d/*

9. Problematiche di Aggiornamento e Disinstallazione

□ Questa sezione era accennata negli appunti grezzi (“presentano dei rischi”) senza dettaglio.

Aggiornare un pacchetto può causare problemi su: 1. **Prerequisiti:** i pacchetti-dipendenza potrebbero dover essere aggiornati a loro volta, rompendo altri software 2. **Configurazione:** nuove versioni possono introdurre format incompatibili con le config già presenti 3. **Dipendenze di altri software:** modifiche alle interfacce rompono software terzi 4. **PATH:** l'ordine di ricerca degli eseguibili può selezionare la versione sbagliata 5. **Loader dinamico:** /etc/ld.so.conf controlla quali librerie .so vengono caricate; ldconfig aggiorna la cache

```
# Verificare quale libreria viene caricata effettivamente:  
ldd /usr/sbin/sshd | grep libz
```

```
# Forzare una versione alternativa:  
export LD_LIBRARY_PATH=/usr/local/lib  
ldd /usr/sbin/sshd | grep libz    # ora punta a /usr/local/lib/libz.so.1
```

Questi rischi esistono anche per la disinstallazione. Il **grafo delle dipendenze** è il valore aggiunto principale del sistema a pacchetti: rende prevedibili gli effetti a cascata.

10. Snap/Flatpak e Container

□ Lorenzo ha annotato “non mi sembra importante per il mio esame”
— incluso per completezza, non è oggetto di esercizi pratici.

Snap/Flatpak: pacchetti indipendenti dalla distribuzione, con tutte le dipendenze incluse. Utili per software particolarmente longevo o basato su funzioni non standard.

Container (Docker, LXC): invece di isolare solo il pacchetto, si isola l’intero ambiente di esecuzione di un processo. Il kernel Linux fornisce tre meccanismi che i container combinano: - **cgroups**: misurano e limitano le risorse (CPU, RAM, I/O, rete) - **namespaces**: ogni processo vede istanze virtuali delle risorse (mnt, pid, net, ipc, uts, user) - **union-capable filesystems**: combinano layer di filesystem (base dei container)

Esercizi sulla VM — Risultati

Es. 1 — Esplorazione base □

```
sudo apt update
apt list --upgradable 2>/dev/null | wc -l
dpkg -l | wc -l      # → 325 pacchetti installati
```

Domanda: cosa fa 2>/dev/null | wc -l?

Sono due pezzi distinti: - 2>/dev/null: reindirige lo stderr (file descriptor 2) verso /dev/null, che scarta tutto. apt list --upgradable stampa un messaggio di warning su stderr (“WARNING: apt does not have a stable CLI interface”); con 2>/dev/null lo eliminiamo per non contare quella riga. - | wc -l: conta le righe dello stdout. Sì — wc -l conta le righe.

Senza 2>/dev/null il conteggio sarebbe sfalsato di 1 dal warning.

Es. 2 — Installare e ispezionare □

```
sudo apt install tree
dpkg -L tree      # → lista file installati
apt show tree     # → descrizione completa
tree /etc/systemd/system/
```

Domanda: come funziona dpkg? Cosa fa -L?

dpkg è il **tool di basso livello** per la gestione dei pacchetti Debian — opera direttamente sul database locale in `/var/lib/dpkg/`. apt si appoggia a dpkg sotto il cappello.

Il flag `-L` (List) mostra tutti i file che un pacchetto ha installato sul filesystem. È utile per sapere dove si trovano i binari, le configurazioni e i man page di un programma. Altri flag utili: `-s <nome>`: mostra lo stato del pacchetto (installato, versione, dipendenze) `-S /path/file`: risale dal file al pacchetto che lo ha installato (inverso di `-L`) `-i file.deb`: installa un pacchetto da file locale

Es. 3 — Cercare e ispezionare prima di installare □

```
apt search htop
apt show htop
sudo apt install htop
apt-cache showpkg htop
```

Domanda: apt search mostra risultati “correlati” — non capisco cosa sono e perché li vede.

`apt search <keyword>` cerca la keyword nel **nome** e nella **descrizione** di tutti i pacchetti nell’indice locale. Se cerchi “htop” trovi sia htop sia iotop, atop, btop ecc. — non perché siano dipendenze di htop, ma perché le loro descrizioni contengono “htop” o un termine correlato. Non sono installati: `apt search` mostra solo pacchetti disponibili nel repo.

Domanda: apt show htop — non lo avevamo già visto?

Sì, l’hai usato su `tree` nell’Es. 2. È lo stesso comando — qui lo stai applicando su htop prima di installarlo, come buona prassi per verificare dipendenze e dimensione prima di procedere.

Domanda: htop ha chiesto autorizzazione Y/n, tree no — perché?

Caso leggermente diverso: `tree` probabilmente era stato già installato in precedenza (o aveva zero dipendenze nuove). htop ha dipendenze aggiuntive che non erano presenti — apt chiede conferma prima di scaricare e installare pacchetti extra. Con `apt install -y` si bypassa la domanda (utile negli script).

Domanda: apt-cache showpkg htop — non capisco la struttura dell’output.

L’output ha tre sezioni: 1. **Package:** — nome e versioni disponibili 2. **Versions:** — versioni con le dipendenze fisiche di ciascuna 3. **Reverse Depends:** — quali altri pacchetti dipendono da htop (chi lo

usa come dipendenza) 4. **Dependencies:** — le dipendenze nella versione installata 5. **Provides:** — nomi alternativi con cui il pacchetto si presenta

Quello che ti interessa di solito è la sezione **Reverse Depends** (per capire cosa si rompe se rimuovi il pacchetto) e il repo di origine (visibile nella riga delle versioni).

Es. 4 — Risalire dal file al pacchetto □

```
dpkg -S /bin/ls          # → coreutils: /bin/ls
dpkg -S /usr/bin/ssh     # → openssh-client: /usr/bin/ssh
dpkg -L openssh-client   # → elenco file del pacchetto
```

Domanda: dpkg -S /bin/ls ha risposto coreutils: /bin/ls — cosa significa?

Significa che il file /bin/ls è stato installato dal pacchetto coreutils. Il formato è <nome-pacchetto>: <path-file>. coreutils (Core Utilities) è il pacchetto che contiene i comandi Unix fondamentali: ls, cp, mv, rm, cat, echo, mkdir ecc. — tutti in un unico pacchetto.

Es. 5 — Dipendenze dinamiche con ldd □

```
ldd /usr/sbin/sshd      # lista librerie dinamiche
ldd /usr/sbin/sshd | wc -l # quante
ldd /bin/ls              # confronto con binario semplice
```

Es. 6 — Rimozione □

```
sudo apt remove tree
which tree          # → errore (non trovato)
sudo apt autoremove
```

Es. 7 — Esplora /etc/apt/sources.list □

```
cat /etc/apt/sources.list
ls /etc/apt/sources.list.d/
ls /etc/apt/trusted.gpg.d/
```

Domanda: cosa mi restituisce sources.list?

Il file contiene le righe di configurazione dei repository abilitati. Ogni riga attiva inizia con deb e ha questa struttura:

```
deb http://deb.debian.org/debian bookworm main contrib non-free-
firmware
```

- **deb**: tipo (pacchetti binari; deb-src sarebbe sorgenti)
- **URL**: indirizzo del mirror
- **bookworm**: nome in codice della versione Debian (la nostra VM)
- **main contrib non-free firmware**: componenti abilitati (main = solo software libero; contrib = dipende da non-libero; non-free = software proprietario)

Le righe commentate con # sono disabilitate. `sources.list.d/` contiene frammenti aggiuntivi — di solito aggiunti da software di terze parti. `trusted.gpg.d/` contiene le chiavi GPG che firmano i repo.

Riepilogo

- Un **pacchetto** è un file che contiene binari precompilati + metadati (dipendenze, versione, architettura) + script di pre/post-installazione. Debian usa `.deb`; RedHat usa `.rpm`.
 - Il **repository** è una raccolta indicizzata di pacchetti; `apt update` sincronizza l'indice locale. La lista dei repo è in `/etc/apt/sources.list` e `/etc/apt/sources.list.d/`.
 - `dpkg` opera sul singolo file `.deb`; `apt` gestisce il ciclo completo incluse le dipendenze automaticamente.
 - Il **grafo delle dipendenze** è il valore aggiunto principale: rende prevedibile cosa si installa, cosa si rompe all'aggiornamento, cosa rimane orfano alla rimozione.
 - L'**autenticità** è garantita da firme GPG: le chiavi di verifica dei repo vanno in `/etc/apt/trusted.gpg.d/`. Un repo non firmato è un rischio di *software injection*.
-

Connessioni

- **Con 3A (systemd)**: i servizi avviati da `systemd` sono programmi installati tramite pacchetti. `apt install nginx` crea automaticamente la unit `systemd`; `dpkg -L nginx` mostra dove è finito il file unit.
- **Con Security (S1 — Enumerazione)**: `Nmap` identifica i servizi in ascolto; ogni servizio è un pacchetto installato. `dpkg -l` e `apt list --installed` sono fondamentali per l'inventario della superficie d'attacco. La *software injection* è direttamente rilevante per supply chain attacks.